Theory Assignment - 3

Frontend (JavaScript) | Deadline: 22/Oct/2026

1. What is the difference between frontend and backend JavaScript?

Frontend JavaScript runs in the user's browser and controls what the user sees and interacts with. **Backend JavaScript** runs on a server (usually with Node.js) and handles data, business logic and databases.

- **Environment:** Frontend runs in the browser (Chrome, Firefox); backend runs in Node.js.
- **Access:** Frontend can use the DOM, `window`, `localStorage`; backend can use the file system, databases, OS and network sockets.
- **Purpose:** Frontend handles UI, events, animations and form validation; backend handles APIs, authentication and data storage.
- **Examples:** Frontend - React, Vue, plain DOM scripts; Backend - Express, NestJS.

2. What is DOM?

The **Document Object Model (DOM)** is a tree-like representation of an HTML page created by the browser. Every element, attribute and piece of text becomes a *node* that JavaScript can read, change, add or remove, so the page can update dynamically without reloading.

```
const title = document.getElementById("title");   // select
title.textContent = "Hello DOM";                   // change text
const p = document.createElement("p");             // create
document.body.appendChild(p);                      // add to page
```

3. What is event in JavaScript?

An **event** is an action or occurrence in the browser that JavaScript can respond to, such as a click, key press, form submit, mouse move or page load. We attach an **event listener** (handler function) to an element, and it runs when the event happens.

```
const btn = document.querySelector("#btn");
btn.addEventListener("click", function (e) {
  console.log("Button clicked!", e.target);
});
```

4. What is event loop in JavaScript?

JavaScript is single-threaded (one call stack), yet it handles asynchronous work like timers, network requests and events. The **event loop** makes this possible. It constantly checks whether the **call stack** is empty; if so, it moves the next task from the queue to the stack.

- **Call Stack:** where functions currently execute.
- **Web APIs:** browser features (setTimeout, fetch, DOM events) that work outside the stack.
- **Microtask Queue:** promise callbacks (`.then`, `async/await`); has higher priority.
- **Callback (Macrotask) Queue:** setTimeout, setInterval and event callbacks.

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
// Output: 1, 4, 3, 2
```

5. What is hoisting in JavaScript?

Hoisting is JavaScript's behavior of moving declarations to the top of their scope during the compilation phase, before code runs.

- `var` is hoisted and initialized as `undefined`.
- Function declarations are fully hoisted (can be called before they are defined).
- `let` and `const` are hoisted but stay in the *Temporal Dead Zone* until declared, so using them early throws a ReferenceError.

```
console.log(a);   // undefined
var a = 5;

sayHi();          // "Hi" - works
function sayHi() { console.log("Hi"); }

console.log(b);   // ReferenceError
let b = 10;
```

6. What is event bubbling?

Event bubbling is the process where an event triggered on an inner element first runs its own handler, then propagates upward through its parent elements up to the document. The opposite direction (top to bottom) is called *capturing*. Bubbling can be stopped with `event.stopPropagation()`, and it enables **event delegation** (one listener on a parent handling many children).

```
// <div id="parent"><button id="child">Click</button></div>
parent.addEventListener("click", () => console.log("Parent"));
child.addEventListener("click", () => console.log("Child"));
// Clicking the button prints: Child, then Parent
```

7. Difference between named function, anonymous function and arrow function?

```
// Named function
function add(a, b) { return a + b; }

// Anonymous function (no name, often stored in a variable)
const sub = function (a, b) { return a - b; };

// Arrow function (short syntax, ES6)
const mul = (a, b) => a * b;
```

- **Named function:** has its own name, is hoisted (declarations), easier to debug and can call itself recursively.
- **Anonymous function:** has no name; used as callbacks or assigned to variables; not hoisted when assigned with var/let/const.
- **Arrow function:** shorter syntax with implicit return; **does not have its own** `this` (uses the surrounding one), no `arguments` object, and cannot be used as a constructor.

8. Explain for-of and for-in loop?

for...in loops over the enumerable **keys (property names)** of an object. **for...of** loops over the **values** of iterable objects such as arrays, strings, Maps and Sets.

```
const arr = ["a", "b", "c"];
```

```
for (let i in arr) console.log(i);   // 0, 1, 2   (indexes/keys)
for (let v of arr) console.log(v);   // a, b, c   (values)

const user = { name: "Ravi", age: 20 };
for (let key in user) console.log(key, user[key]);
```

Tip: use for-in for objects and for-of for arrays; for-of does not work directly on plain objects.

9. Explain any five array methods?

- **push() / pop():** `push` adds an element at the end; `pop` removes the last element.
- **map():** creates a new array by applying a function to every element.
- **filter():** creates a new array with elements that pass a condition.
- **reduce():** reduces the array to a single value (sum, product, etc.).
- **forEach():** runs a function for each element (returns nothing).

```
const nums = [1, 2, 3, 4, 5];
nums.push(6);                                  // [1,2,3,4,5,6]
nums.pop();                                    // [1,2,3,4,5]
nums.map(n => n * 2);                          // [2,4,6,8,10]
nums.filter(n => n % 2 === 0);                 // [2,4]
nums.reduce((sum, n) => sum + n, 0);           // 15
nums.forEach(n => console.log(n));             // prints each number
```

10. What is callback method?

A **callback** is a function passed as an argument to another function, to be executed later (after a task finishes or an event occurs). Callbacks are the basis of asynchronous programming in JavaScript, used in timers, events and array methods. Deeply nested callbacks lead to "callback hell", which is solved using Promises and async/await.

```
function greet(name, callback) {
  console.log("Hello " + name);
  callback();
}
greet("Ravi", function () {
  console.log("Callback executed");
});

setTimeout(() => console.log("Runs after 1 second"), 1000);
```